

94. 二叉树的中序遍历

题目描述

给定一个二叉树的根节点 `root`，返回它的 **中序遍历**。

示例 1:

输入: `root = [1,null,2,3]`

输出: `[1,3,2]`

示例 2:

输入: `root = []`

输出: `[]`

示例 3:

输入: `root = [1]`

输出: `[1]`

提示:

- 树中节点数目在范围 $[0, 100]$ 内
- $-100 \leq \text{Node.val} \leq 100$

进阶: 递归算法很简单，你可以通过迭代算法完成吗？

解题思路

二叉树的中序遍历顺序是：**左子树** -> **根节点** -> **右子树**。

方法一：递归

这是最直观的方法。

1. **终止条件:** 如果当前节点为空 (`null`)，则直接返回。
2. **递归左子树:** 对当前节点的左子节点调用递归函数。
3. **访问根节点:** 将当前节点的值添加到结果列表中。
4. **递归右子树:** 对当前节点的右子节点调用递归函数。

复杂度分析: * 时间复杂度: $O(N)$, N 为节点数。* 空间复杂度: $O(H)$, H 为树高, 最坏 $O(N)$, 平均 $O(\log N)$ 。

方法二：迭代（使用栈）

使用栈来模拟递归调用过程。

1. 初始化空栈 `stack` 和空列表 `result`。
2. 初始化当前节点 `curr` 指向根节点 `root`。
3. **循环条件:** 当 `curr` 不为空或 `stack` 不为空时:
 - **处理左子树:** 当 `curr` 不为空时, 将 `curr` 压入栈中, `curr` 移至其左子节点 (`curr = curr.left`)。

- **处理根节点和右子树:** 当 curr 为空时, 从栈中弹出节点 node, 将其值加入 result, curr 移至 node 的右子节点 (curr = node.right)。

4. 返回 result。

复杂度分析: * 时间复杂度: $O(N)$ 。* 空间复杂度: $O(H)$, 最坏 $O(N)$, 平均 $O(\log N)$ 。

代码实现 (Python)

```
# Definition for a binary tree node.
class TreeNode:
    #     def __init__(self, val=0, left=None, right=None):
    #         self.val = val
    #         self.left = left
    #         self.right = right

from typing import List, Optional

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    # 方法一: 递归
    def inorderTraversalRecursive(self, root: Optional[TreeNode]) -> List[int]:
        result = []
        def traverse(node):
            if not node:
                return
            traverse(node.left)
            result.append(node.val)
            traverse(node.right)
        traverse(root)
        return result

    # 方法二: 迭代
    def inorderTraversalIterative(self, root: Optional[TreeNode]) -> List[int]:
        result = []
        stack = []
        curr = root
        while curr or stack:
            # 不断往左子树方向走, 每走一次就将当前节点保存到栈中
            while curr:
                stack.append(curr)
                curr = curr.left
```

```

        # 当前节点为空, 说明左边走到头了, 从栈中弹出节点并保存
        # 然后转向右边节点, 继续上面整个过程
        curr = stack.pop()
        result.append(curr.val)
        curr = curr.right
    return result

# 通常面试时会让你选择一种方法实现
def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
    # 这里可以选择返回递归或迭代版本
    # return self.inorderTraversalRecursive(root)
    return self.inorderTraversalIterative(root)

```

面试回答技巧

1. **清晰阐述思路**: 首先明确中序遍历的定义 (左-根-右)。
2. **从递归入手**: 通常先给出递归解法, 解释递归步骤和终止条件。
3. **分析复杂度 (递归)**: 讨论时间 $O(N)$ 和空间 $O(H)$ 复杂度。
4. **应对进阶 (迭代)**: 解释为何需要迭代以及如何使用栈模拟。
5. **阐述迭代思路**: 详细解释栈的操作过程, 可画图辅助。
6. **分析复杂度 (迭代)**: 讨论时间 $O(N)$ 和空间 $O(H)$ 复杂度。
7. **处理边界条件**: 提及空树情况。
8. **代码风格**: 保持代码清晰、规范。

示例回答框架:

“对于二叉树的中序遍历, 核心思想是按照‘左子树-根节点-右子树’的顺序访问节点。

最自然的方法是递归。我们可以定义一个递归函数, 先递归调用左子树, 然后访问当前节点, 最后递归调用右子树。终止条件是节点为空。时间复杂度 $O(N)$, 空间复杂度平均 $O(\log N)$, 最坏 $O(N)$ 。

如果需要迭代实现, 我们可以使用一个栈。用指针 `curr` 从根开始, 不断将 `curr` 及其左子节点压栈, 直到最左端。然后出栈访问节点, 并将 `curr` 指向弹出节点的右子树, 重复此过程。时间复杂度 $O(N)$, 空间复杂度平均 $O(\log N)$, 最坏 $O(N)$ 。这种方法能避免递归栈溢出。

实现时需注意空树边界。”